Guia de estudo – SB (CIC0104) – Prova-T "Softwares de Sistemas"

Guia único montado a partir das três provas resolvidas, com as respostas **reconciliadas contra os slides**. Para ver o enunciado completo e o comentário longo de uma questão, abra o guia individual:

- 01-ProvaSB_teoria-resolvida.md: prova de outro aluno (aqui chamada **P1**).
- 02-SB_Prova_Sistemas-EU-resolvida.md: Prova-T 21/2 com **as suas respostas** (aqui **PT**; seus erros estão na seção 5).
- 03-SB_Prova_Sistemas-resolvida.md: a mesma Prova-T 21/2, respondida por outro aluno (mesma numeração **PT**).

Nenhuma resposta aqui é gabarito oficial. Quando há dúvida, a seção 8 diz o grau de confiança.

Notação dos slides (número impresso no slide): `7-1 s7` = `7-teoria-1 de 2.pdf` , slide 7 (= página 7); `7-2 s49` = `7-teoria-2 de 2.pdf` , slide 49 (= página 17); `8-1 s9` = `8-so-red-1 de 2.pdf` , slide 9 (= página 9); `8-2 s20` = `8-so-red-2 de 2.pdf` , slide 20 (= página 7); `9 s6` = `9-boot-so-red.pdf` , slide 6.

1. Como usar este guia e regras da prova

A descoberta mais importante

P1 e PT são praticamente a mesma prova. Os 22 itens V/F são os mesmos, em outra ordem, e as 4 discursivas são idênticas. Muda só **uma palavra** em alguns itens, o que inverte a resposta:

Item	P1	PT	Efeito
Saída do ligador	"é executável" (Q11) **V**	"não é executável" (Q21) **F**	inverte
Round-robin	"evitar **starvation**" (Q15) **V**	"evitar **deadlock**" (Q20) **F**	inverte
Fragmentação	"baseada em **segmentos**" (Q21) **V**	"baseada em **páginas**" (Q12) **F**	inverte

Item	P1	PT	Efeito
Estrutura do montador	"síntese de código (... absoluto)" (Q9)	"passo 2 (... absoluto)" (Q9)	os dois **F**
Tipos de ligador	"podem ser **apenas** dos tipos" (Q13)	sem "apenas" (Q13)	os dois **V**

Estratégia: domine os 22 itens (seção 3) e as suas versões "viradas", e decore as 4 respostas-modelo (seção 4). Na prova, procure a palavra trocada antes de qualquer outra coisa.

Ordem de leitura sugerida: seção 2 (o que priorizar), seção 3 (banco), seção 4 (discursivas), seção 5 (seus erros). Na véspera, só a seção 7. A seção 8 serve para decidir os itens duvidosos.

Regras da prova (cabeçalho da Prova-T 21/2)

- Preencher **V**, **F** ou **NS** (não sei). Parte 1 vale 7,0 (22 × 0,32); Parte 2 vale 1,0 + 1,0 + 1,0 + 2,0.
- **Duas erradas anulam uma certa**, ou seja, cada erro custa −0,16. **Em branco conta como errada** (−0,16). NS vale 0.
- **Justificar todo item marcado F.** Um F sem justificativa corre o risco de não ser aceito.
- Uma marcação só por item. "F/NS" (como na sua Q6) é inválida e tende a contar como erro.
- "Mantenha o formato de tabela": não mude o layout.

Quando vale marcar NS

Sua chance de acertar	Valor esperado de responder	Decisão
50% (chute puro em V/F)	0,5 × 0,32 − 0,5 × 0,16 = **+0,08**	responda
40%	+0,032	responda
33%	**0**	indiferente
abaixo de 33%	negativo	NS

Consequências práticas:

1. **Nunca deixe em branco.** Branco é −0,16; NS é 0.
2. Em V/F, até o chute puro tem valor esperado positivo. **NS só compensa quando você desconfia que o seu palpite está errado** (o item parece pegadinha e você não sabe para que lado).

3. **Se você acha que é F mas não consegue escrever nenhuma justificativa**, o F pode não valer. Nesse caso marque V se ainda achar V plausível (mais de 1/3), ou NS.
4. Ao marcar F, a justificativa deve **apontar o trecho errado e dar a versão correta** em uma linha. Ex.: "O carregador absoluto fica em memória **não volátil** (ROM/flash)."
5. Não escreva justificativa em item marcado V (o outro aluno da PT fez isso na Q17, o que é contraditório).

2. Mapa de assuntos por frequência

Assunto	P1	PT	Itens (P1 / PT)	Prioridade
von Neumann: componentes, ciclo, localidade, gargalo	4 obj + D1	4 obj + D1	1, 2, 3, 4 / 1, 2, 4, 14	**máxima**
Código absoluto, relocação, carregadores, boot	4 obj + D2	4 obj + D2	5, 6, 7, 14 / 5, 7, 8, 15	**máxima**
Processos: escalonamento, deadlock, starvation, região crítica	4 obj + D4 (1,5 pt)	4 obj + D4 (1,5 pt)	15, 16, 17, 18 / 16, 17, 19, 20	**máxima** (D4 vale 2,0)
Gerência de memória (real/virtual, paginação, segmentação)	3 obj + D3 + parte da D4	3 obj + D3 + parte da D4	19, 20, 21 / 10, 11, 12	**máxima**
Montador	3	3	8, 9, 10 / 6, 9, 22	alta
Ligador	3	3	11, 12, 13 / 3, 13, 21	alta
Tipos e estrutura de S.O. (batch × time sharing)	2 obj + D4	2 obj + D4	19, 20 / 10, 11	alta (cai junto com memória)
E/S, interrupção, page fault	1 obj + 1 palavra	1 obj	22, extra / 18	média
Boot do Windows XP (slide 9: POST, MBR, NTLDR)	0	0	só FFFF:0000 aparece	baixa (só os fatos da seção

Assunto	P1	PT	Itens (P1 / PT)	Prioridade
				7)

Pontos por bloco (por prova): von Neumann ≈ 1,28 + 1,0; carregadores ≈ 1,28 + 1,0; processos ≈ 1,28 + 1,5; memória ≈ 0,96 + 1,0 + 0,5. **As discursivas valem 5,0 de 10**, e são sempre as mesmas 4.

3. Banco de questões consolidado por assunto

3.0 Critério para enunciados incompletos (decisão da reconciliação)

O problema: os guias marcaram F o item de von Neumann (P1-Q1 / PT-Q1) por omitir os barramentos, mas V o item do ciclo (P1-Q2 / PT-Q14), que omite o incremento do PC. Parece contraditório. Não é, se o critério for este:

> **Um item é F quando contém uma afirmação errada, OU quando se apresenta como enumeração fechada de uma lista do slide ("é caracterizada por...", "é composto por...", "são dos tipos...", "seguindo as etapas...") e deixa de fora um elemento dessa lista.**
> **Um item que descreve um processo ou fato, sem pretender listar tudo, e em que toda afirmação está correta, é V, mesmo que não cite outros detalhes do slide.**

Aplicação:

Item	O que diz	Omissão	Algo errado?	Resposta
von Neumann (P1-Q1, PT-Q1)	"é **caracterizada por** CPU, memória e periféricos"	os **barramentos**, 1 dos 4 itens da lista de 7-1 s5	sim, secundário: "memória **RAM**" restringe o slide ("memória"; o boot fica em ROM)	**F**
Ciclo e PC (P1-Q2, PT-Q14)	"segue o ciclo busca–decodificação–busca de operandos–execução; o carregador atribui a	o incremento do PC, que é **outro marcador** em 7-1 s7, e não parte do	nada	**V**

Item	O que diz	Omissão	Algo errado?	Resposta
	PC o entry point e inicia o ciclo"	ciclo; as 4 fases estão completas		

Evidência de que é o padrão do professor: o item do carregador de relocação (P1-Q14 / PT-Q7) também é incompleto ("lê o tamanho do programa", e o slide 7-2 s49 diz "lê o tamanho **e o entrypoint**"), e os três guias e os dois alunos marcaram V. O montador (P1-Q8 / PT-Q22) omite a "função básica" do slide 17 e é V. Ou seja: **omitir um detalhe não torna falso; tirar um elemento de uma lista definidora, sim.**

Ressalva honesta: dois alunos da PT (você e o outro) marcaram F na Q14, e não há gabarito. A confiança no V é **média** (cerca de 65%). Mesmo assim, V tem valor esperado maior: com 65% de chance, V vale +0,15 e F vale +0,01.

O que fazer na prova:

1. Enumeração fechada com item faltando: **F**, e a justificativa cita o item que falta.
2. Descrição de processo com tudo correto: **V**. Não justifique F com "não fala de X".
3. Se ficar realmente dividido, **não marque NS**: escolha a leitura acima (o chute já tem valor esperado positivo).

3.1 Arquitetura de von Neumann e localidade (7-1 s5–s12)

(a) Componentes – P1-Q1, PT-Q1 → **F**

- Variação vista: "caracterizada por CPU (ULA, UC, registradores, PC), memória (RAM...) e periféricos". As duas provas têm o mesmo texto.
- **Justificativa:** "Faltam os **barramentos** (dados, endereço e controle). Além disso, a memória não é necessariamente RAM (o boot está em ROM)."
- Pegadinha: lista do slide com um item a menos. A sua justificativa na PT estava perfeita. Evite a do aluno de P1 ("a RAM não necessariamente será a memória usada"), que pega só o detalhe secundário.
- Decore: **4 componentes** (CPU, memória, E/S, barramentos); CPU = ULA, UC, registradores, PC; **3 barramentos** (dados, endereço, controle).

(b) Ciclo e entry point – P1-Q2, PT-Q14 → **V** (confiança média; ver 3.0)

- Fonte: 7-1 s7: "Ciclo busca – decodificação – busca de operandos – execução e PC; PC ← Entry point; PC é incrementado a cada ciclo de busca".
- Variantes falsas possíveis: ordem do ciclo trocada; "o **montador**/ligador atribui o entry point ao PC" (quem faz é o **carregador**); "PC é decrementado".

(c) Execução rápida e localidade – P1-Q3, PT-Q2 → **V**

- É a cópia de 7-1 s7. A causalidade do "devido ao fenômeno" está um pouco invertida (é o armazenamento adjacente que **gera** a localidade), mas o texto é o do slide. Os três guias marcaram V.

(d) Gargalo de von Neumann – P1-Q4, PT-Q4 → **F**

- Item: "gargalo entre CPU e periféricos de I/O ... cache e pipelining".
- **Justificativa:** "Cache e pipelining reduzem o gargalo **CPU × memória**. O gargalo **memória × E/S** se reduz com blocagem/bufferização, DMA e canais de E/S."
- Tabela a decorar (7-1 s8):

Gargalo	Técnicas
CPU × memória	cache; pipelining (múltiplos estágios)
memória × periféricos de E/S	blocagem e bufferização; DMA; canais de E/S

- Pegadinha: troca das colunas. A justificativa do aluno de P1 ("além dos citados, necessita DMA...") está **errada**, porque admite que cache ajuda no gargalo de E/S.

3.2 Montador (7-1 s17–s31)

(a) Definição e saída – P1-Q8, PT-Q22 → **V**

- Cópia de 7-1 s17: Assembly → **código objeto relocável**, se o programa estiver correto; saída = binário relocável + tabela de endereços relocáveis + lista de referências externas (símbolos não resolvidos).
- Variantes falsas prováveis: "gera código **absoluto**", "gera código **executável**", "resolve as referências externas" (isso é o ligador), "gera o código mesmo com erro de sintaxe" (s17: com erro, não gera).

(b) Estrutura lógica – P1-Q9 ("síntese de código: otimização e geração de código absoluto"), PT-Q9 ("passo 2: otimização e geração de código absoluto") → **F** nas duas

- **Justificativa:** "O montador gera código objeto **relocável**, não absoluto (7-1 s17, s18, s31)."
- Pegadinha: "absoluto" escondido no fim de uma frase longa e correta. O aluno de P1 marcou NS; dava para marcar F com segurança.
- Detalhe (do guia 03): a estrutura **lógica** é análise (léxica → sintática → semântica → código intermediário) + síntese (otimização opcional + geração); "passos" é a estrutura **física** (leituras do fonte, 7-1 s18: pré-passo, passo 1, passo n, passo final). Não muda a resposta.

(c) Otimizações na síntese – P1-Q10, PT-Q6 → **F** (confiança média, cerca de 70%)

- Item: "As otimizações **algorítmicas** de código que podem ser realizadas na síntese são de tempo de execução ou de alocação de memória."
- Fonte: 7-1 s31: "Otimiza o código intermediário... **A otimização é do código e não das funções implementadas ao nível do código do programador**."
- **Justificativa:** "A síntese otimiza o **código intermediário**; não faz otimização algorítmica (das funções do programador)."
- Os três guias concordam em F. Os dois alunos de fora marcaram V; você marcou "F/NS" com justificativa contrária ao slide.

3.3 Ligador (7-2 s34–s50)

(a) Saída – P1-Q11 ("é executável") → **V**; PT-Q21 ("**não** é executável") → **F**

- Fonte: 7-2 s34: saída = "arquivo binário com código de máquina relocável **executável**" + endereços relocáveis.
- **Justificativa (PT):** "O arquivo gerado pelo ligador **é executável** (e continua relocável: o carregador de relocação ajusta os endereços na carga)."
- Pegadinha: o final da frase alterna entre as provas. Lembre que a saída é ao mesmo tempo **executável e relocável**.

(b) Etapas – P1-Q12, PT-Q3 → **V**

- Cópia de 7-2 s40, na ordem: **ler e relocar P → atualizar a tabela global com os símbolos internos (públicos) de P → concatenar P → resolver as referências externas de P**.
- Variantes falsas prováveis: ordem trocada; "símbolos **externos** de P na tabela global" (a tabela global guarda os **públicos/internos**, 7-2 s47); "relocação dinâmica" (é **estática**: k = size(P1), size(P1,P2)..., 7-2 s43).

(c) Tipos de ligador – P1-Q13 ("podem ser **apenas** dos tipos..."), PT-Q13 (sem "apenas") → **V** (confiança média-alta, cerca de 75%)

- Os slides intitulados "Tipos de Ligadores" listam exatamente três: **Linkage Editor** (liga antes da carga e grava o módulo de carga, reutilizável) e **Linking Loader/Binder** (liga e carrega, **não gera arquivo**), em 7-2 s48; e **Dynamic linking** (DLL, em tempo de execução), em 7-2 s50.
- Riscos: o "apenas" (P1) e o termo "carregador de função" (que não está no slide; mas 7-2 diz que "o carregador é quem se encarrega de resolver a relocação das funções de bibliotecas"). A classificação por número de passos (um/dois) é **outra classificação**, não outro tipo.
- Variante falsa provável: trocar as descrições (Linking Loader "gera arquivo reutilizável").

3.4 Código absoluto, relocação, carregadores e boot (7-1 s13–s16, 7-2 s38, s49, 9 s6)

(a) Código absoluto "não é mais usado" – P1-Q5, PT-Q8 → **F**

- **Justificativa:** "Código absoluto ainda é usado no boot: ROM BIOS (FFFF:0000) e carregador absoluto (7-1 s13, s15)." A sua ("dependendo do sistema, ainda pode ser utilizado") está certa, mas genérica: cite o boot.

(b) Código relocável e bytecode Java – P1-Q6, PT-Q15 → **F** (confiança média, cerca de 70%)

- A primeira parte é verdadeira: "Relocação estática viabiliza a construção de bibliotecas de código objeto" (7-1 s14, s23).
- O **exemplo** é falso: bytecode Java não é código objeto relocável (código de máquina + lista de endereços relocáveis). É código intermediário de **máquina virtual**, e roda em qualquer computador por causa da **JVM**, não da relocação.
- **Justificativa:** "Bytecode Java não é código objeto relocável; sua portabilidade vem da JVM, não da relocação."
- Os três guias concordam em F. O aluno de P1 marcou V; você e o outro aluno da PT marcaram NS.
- Pegadinha: primeira metade verdadeira + exemplo errado = F.

(c) Carregador absoluto em memória volátil – P1-Q7, PT-Q5 → **F**

- **Justificativa:** "O carregador absoluto fica em memória **não volátil** (ROM/flash), em posição fixa e conhecida (7-1 s15)."
- A frase toda é do slide, exceto a palavra "volátil". Variante verdadeira provável: "não volátil" → V. Também é o **IPL** no mainframe e é ativado pelo botão.

(d) Algoritmo do carregador de relocação – P1-Q14, PT-Q7 → **V**

- 7-2 s49: lê o tamanho (e o entrypoint) → pede ao S.O. um **bloco contíguo** → carrega relocando os endereços relocáveis (endereço = offset + α, constante k) → transfere o controle a partir do entrypoint (PC ← entrypoint).
- Variantes falsas prováveis: "blocos não contíguos", "a relocação é feita durante a execução" (isso é relocação **dinâmica**, registrador de base/paginação), "quem reloca é o S.O.".
- Pergunta do slide: "Qual o problema com esse esquema?" Ver 4.5.

3.5 Tipos de S.O. e gerência de memória "melhor" (8-1 s4, s13–s15; 8-2 s16–s23)

Par trocado de propósito: o professor escreveu os dois itens com as associações invertidas. As associações corretas são:

- **batch** ↔ jobs CPU-intensivos, não interativos ↔ **partições** (contíguo);
- **time sharing** ↔ interativo/E/S, time slice ↔ **swapping** (troca via interrupção de time slice, 8-2 s18).

(a) Batch + paginação-segmentação – P1-Q19, PT-Q10 → **F** (confiança média-alta, cerca de 75%)

- **Justificativa:** "Paginação-segmentação é gerência de memória **virtual** (8-2 s23), não real. Para batch (jobs CPU-intensivos) bastam **partições**."
- Os três guias concordam. Nenhum slide diz literalmente "melhor para batch = X". O argumento mais sólido é o real × virtual.

(b) Time sharing + "CPU intensiva" + partições – P1-Q20, PT-Q11 → **F** (confiança alta)

- **Justificativa:** "Em time sharing as transações são **interativas/E/S intensivas** (CPU intensivo é o batch), e a gerência real adequada é o **swapping** (time slice), não partições."
- Sua justificativa na PT ("Sistema operacional do tipo batch") está certa no conteúdo, mas curta demais para garantir o ponto. Escreva a frase completa.

Real × virtual nos slides (decore; é o que decide estes itens e a D4):

	Contígua	**Não contígua**
Monoprogramação	código absoluto; código relocável (estático)	overlay

	Contígua	Não contígua
Multiprogramação	**partições; swapping; registrador de base** (relocação dinâmica)	**paginação; segmentação; paginação-segmentação**

- Títulos "Gerência de Memória **Real**": overlay, partições, swapping, registrador de base (8-2 s16–s19).
- Títulos "Gerência de Memória **Virtual**": paginação, segmentação, segmentação-paginação (8-2 s20–s23).

3.6 Paginação, segmentação e fragmentação (8-1 s13; 8-2 s20–s23)

(a) Fragmentação – P1-Q21 ("baseada em **segmentos** minimiza a fragmentação interna **mas** favorece a proteção de código e a fragmentação externa") → **V**; PT-Q12 ("baseada em **páginas** minimiza a fragmentação interna, favorece a proteção de código e a fragmentação externa") → **F**

- Conferi a P1 no PDF original: é mesmo "segmentos" (o item aparece duas vezes por causa da quebra de página).
- A diferença é **só do enunciado**, e não um erro de agente. Mas o **aluno de P1 errou**: marcou F ("aumenta a fragmentação interna"). E o outro aluno da PT também errou (marcou V).
- **Justificativa (PT):** "A paginação usa blocos de tamanho fixo: **não tem fragmentação externa** e tem fragmentação interna (última página). O descrito é a segmentação."
- Reforço do slide 8-1 s13: blocos de tamanho fixo → "sem fragmentação"; blocos de tamanho variável → "são necessárias técnicas para combater a fragmentação" (pior, melhor e primeiro ajuste).

	Paginação	Segmentação
Bloco	fixo, potência de 2 (poucos KB)	variável, ligado à lógica do programa
Fragmentação interna	**sim** (última página)	mínima
Fragmentação externa	**não**	**sim**

	Paginação	Segmentação
Endereço real	**concatenação** frame ‖ d	**soma** base + d ("a concatenação não pode ser usada")
Proteção	"facilita" (s20)	"facilita **em muito**" (s22)
Custo	menor	maior
Segmentação-paginação	segmento de tamanho variável, mas múltiplo da página; endereço com 3 níveis (segmento, página, deslocamento); 2 tabelas; termina em concatenação; "largamente utilizado hoje" (s23)	

(b) Page fault – P1-Q22, PT-Q18 → **V**

- É o roteiro de E/S por interrupção de 7-1 s9: o processo é **bloqueado**, o supervisor **cria o programa de canal**, o canal faz a E/S e gera interrupção, e o processo volta para os prontos. Se não houver frame livre, o **LRU** escolhe a página a sair (8-2 s20).

3.7 Processos: escalonamento, deadlock, starvation, região crítica (8-1 s9–s10)

(a) Round-robin – P1-Q15 ("pois permite evitar **starvation**") → **V**; PT-Q20 ("pois permite evitar **deadlock**") → **F**

- A diferença é do enunciado. **Round-robin evita starvation, não deadlock.**
- **Justificativa (PT):** "Round-robin evita **starvation** (todos recebem time slice). Deadlock é espera circular por **recursos** e depende da política de alocação de recursos."
- Sua justificativa ("Round-robin evita starvation") está boa. A do outro aluno ("não garante a justiça de atendimento") está **errada**: o round-robin é justamente o algoritmo justo.

(b) "Por isso um bom escalonador deve ser não preemptivo" – P1-Q16, PT-Q19 → **F**

- A primeira frase é do slide 8-1 s9. A conclusão é falsa.
- **Justificativa:** "O escalonamento pode ser preemptivo ou não (8-1 s9). Minimizar o response time exige, em geral, preempção (time slice); sem ela um processo longo monopoliza a CPU."
- Não escreva "deve ser preemptivo" (absoluto demais: o batch pode usar não preemptivo).

(c) Pré-alocação de todos os recursos – P1-Q17, PT-Q16 → **F**

- Item: "para maximizar o throughput **e** evitar deadlock, um processo só entra na fila de prontos com todos os recursos".
- Evita deadlock (quebra a "posse e espera"), mas **reduz** o throughput (recursos presos e ociosos, processos esperando, risco de starvation).
- **Justificativa:** "Evita deadlock, mas não maximiza o throughput: os recursos ficam alocados e ociosos." O aluno de P1 marcou NS; dava para responder.
- Pegadinha: "X **e** Y" é falso se uma das partes for falsa.

(d) Região crítica – P1-Q18, PT-Q17 → **V** (confiança média, cerca de 60%)

- Item: "Região crítica é o trecho do código que objetiva garantir a exclusão mútua de processos." O texto é idêntico nas duas provas, e os três alunos marcaram V.
- **A favor do V:** no slide 8-1 s10, "Região crítica" aparece como subitem do **mecanismo** com que o S.O. controla a alocação de recurso (alocado → bloqueia; livre → aloca). Na terminologia do professor, região crítica é o mecanismo de exclusão mútua. O construto "região crítica" de Brinch Hansen também é um mecanismo que garante exclusão mútua.
- **Contra:** nos livros, região crítica é o trecho que **acessa recurso compartilhado** e que por isso **exige** exclusão mútua; quem a garante são semáforos, mutex e monitores.
- **Erro conceitual a não repetir** (justificativa do outro aluno da PT): "região crítica é onde pode ocorrer um **deadlock**". Errado: sem exclusão mútua, o que ocorre na região crítica é **condição de corrida** (race condition), ou seja, um resultado que depende da ordem de execução dos processos. Deadlock é espera circular por recursos.
- **Recomendação:** marque **V**. Se uma variante disser "região crítica é onde ocorre deadlock" → F, justificando com a condição de corrida.

3.8 E/S e interrupção (7-1 s9–s10; 8-1 s6–s11)

Questão de uma palavra (só P1): "Qual mecanismo retira o processo da fila de bloqueados e o coloca na fila de prontos quando o acesso a disco termina?" → **Interrupção** (confiança média-alta, cerca de 80%)

- 7-1 s9: "Canal gera interrupção → controlador de interrupções desbloqueia processo → processo é enfileirado na fila de prontos". 7-1 s10: "Processo bloqueado acorda ao receber sinal de interrupção".
- O aluno de P1 respondeu "Escalonador", que provavelmente está errado: o **escalonador** move de **pronto** → **executando** (8-1 s9).

- O guia 02 cita "controlador de interrupções", que é o mesmo mecanismo, mas a resposta pede **uma palavra**: **Interrupção**.

Transições de estado (decore):

- executando → bloqueado: pedido de E/S ou de recurso ocupado (chamada ao supervisor);
- bloqueado → pronto: **interrupção** (fim da E/S) / controlador de interrupções;
- pronto → executando: **escalonador**;
- executando → pronto: preempção (fim do time slice, interrupção de relógio).

4. Discursivas (as mesmas 4 nas duas provas)

Escreva em tópicos que espelhem o enunciado, um bloco por pedaço pontuado.

4.1 D1 – Localidade de referência (1,0): "o que é (0,5) e o que causa (0,5)"

O que é (0,5). Tendência de um programa, num curto intervalo de tempo, concentrar seus acessos à memória num pequeno conjunto de endereços:

- **temporal:** um endereço acessado agora tende a ser acessado de novo em breve;
- **espacial:** após acessar um endereço, tende a acessar os vizinhos.

O que causa (0,5). A arquitetura de von Neumann e o modo de programar para ela (7-1 s7):

- dados e instruções em **blocos de memória adjacentes** + **processamento sequencial** (PC incrementado a cada busca) → a próxima instrução está ao lado (espacial);
- **laços** reexecutam as mesmas instruções e variáveis (temporal);
- **vetores, matrizes e registros** contíguos, percorridos em sequência (espacial);
- **pilha** e variáveis locais concentradas no topo; programação modular e estruturada.

Consequência (vale citar): é a localidade que faz funcionarem a **cache**, os buffers, o disk cache e a **paginação/LRU**, e por isso "a execução é rápida".

Cobrado: separar definição de causa; citar temporal **e** espacial; dar causas concretas (não repetir os tipos como causa, que foi o seu erro).

4.2 D2 – Tipos de carregadores (1,0): tipos (0,5), instante da relocação (0,25), o que carrega (0,25)

Carregador	Instante da relocação	O que carrega
Absoluto (ROM/flash, posição fixa, ativado pelo botão; IPL no mainframe)	**não há relocação**: endereços fixos e reais definidos antes	o **início do S.O.** (bootstrap): ROM BIOS em FFFF:0000 → MBR em 0000:7C00 → boot loader → NTLDR
De relocação (estática)	**em tempo de carga, antes da execução**: o tradutor e o ligador geram endereços relativos à base 0, e o carregador soma o endereço α dado pelo S.O. (constante k) a cada endereço da lista de relocáveis (7-1 s14; 7-2 s38, s49)	um **programa executável** (módulo de carga gerado pelo ligador) que está em disco, num bloco **contíguo** da RAM; depois faz PC ← entrypoint

Para enriquecer (pontos extras): **linking loader/binder** (liga, reloca e carrega na carga, sem gerar arquivo; carrega módulos objeto); **ligação dinâmica/DLL** (reloca em tempo de **execução** ou de carga, via tabela *fixup*; carrega rotinas de biblioteca compartilhadas); compare com a relocação **dinâmica** por registrador de base/paginação, feita a **cada acesso** durante a execução.

Cobrado: as **três** partes. Você perdeu o "instante"; o outro aluno perdeu o "o que carrega".

4.3 D3 – Mapeamento lógico → real em ambiente paginado, com exemplo (1,0)

Mecanismo (8-2 s20–s21):

1. Memória física dividida em **frames** de tamanho fixo, potência de 2; o programa é dividido em páginas do mesmo tamanho.
2. O endereço lógico é dividido em **p** (número da página, bits altos) e **d** (deslocamento, bits baixos). Página de 2^n bytes → d tem n bits.
3. O **DAT** (hardware) soma **RegBase** (início da tabela de páginas do processo) **+ p** e lê essa entrada da **tabela de mapeamento de páginas**.
4. A entrada dá o **frame r** (mais bits de presença e de referência/proteção). Se a página não está na memória → **page fault**: interrupção, processo bloqueado, programa de canal traz a página (LRU escolhe a vítima), processo volta aos prontos.

5. **Endereço real = r || d (concatenação)**, não soma: a página é potência de 2 e os frames são alinhados, então basta justapor os bits (mais rápido). Na segmentação é preciso **somar**.

Exemplo: endereço de 16 bits, páginas de 4 KB (2^{12}) → d = 12 bits, p = 4 bits.

p	frame r	presente
0	7	sim
1	3	sim
2	5	sim
3	–	não

Endereço lógico **0x2ABC** → p = 2, d = 0xABC → entrada RegBase + 2 → r = 5 → endereço real = 5 ‖ ABC = **0x5ABC** (= 5 × 4096 + 2748 = 23228). Endereço 0x3010 → p = 3 ausente → **page fault**.

Cobrado: o **exemplo** (obrigatório), a divisão (p, d), a tabela, a **concatenação** e o page fault. **Não confunda** com o registrador de base (8-2 s19: ER = EL + RegBase, esquema contíguo), que foi o seu erro.

4.4 D4 – Projeto de S.O. (2,0): aplicação, carregamento, tipo, estrutura; política de processos (0,5), como evita deadlock (0,5) e starvation (0,5); gerência de memória real (0,5)

Questão aberta: vale a **coerência** entre as escolhas e responder **todos** os itens. Modelo recomendado (todo apoiado nos slides):

- **Aplicação: time sharing**: muitos usuários interativos, transações curtas e de E/S; objetivo: minimizar o **response time** (8-1 s4: uso de time slice).
- **Tipo: multiprogramado**: vários processos na memória; enquanto um espera E/S, outro usa a CPU (7-1 s10).
- **Carregamento: contíguo**, cada processo num bloco, com relocação dinâmica por registrador de base (coerente com o swapping, na mesma célula da tabela de 8-2 s15).
- **Estrutura: hierárquica (camadas)**: o nível 0 (kernel) trata interrupções e escalonamento; cada nível usa os inferiores; excelente abstração (8-2 s25).
- **Política de processos (0,5): round-robin preemptivo com time slice** (com prioridades + aging, opcional). Justificativa: a preempção por interrupção de relógio impede que um processo monopolize a CPU e mantém baixo o response time dos interativos.

- **Deadlock (0,5):** o escalonamento da CPU **não** evita deadlock; é preciso uma regra de alocação de recursos que quebre uma das condições (exclusão mútua, posse e espera, não preempção, espera circular):
  - **ordenação dos recursos**: cada processo só pede recursos em ordem crescente de número, o que impede a **espera circular**;
  - acesso a recursos compartilhados por **região crítica**: recurso ocupado → processo bloqueado (8-1 s10);
  - (alternativas: pré-alocação, que reduz o throughput; algoritmo do banqueiro; detecção e recuperação).
- **Starvation (0,5):** no round-robin a fila é circular: com N processos e fatia q, cada um espera no máximo (N−1)·q, e ninguém fica para trás. Se houver prioridades, **aging** (a prioridade cresce com a espera). Filas de recursos em FIFO.
- **Gerência de memória real (0,5): swapping**: troca processos entre memória e disco via interrupção de time slice (8-2 s18), com **registrador de base/DAT** para o processo voltar em outra posição (8-2 s19). *Complemento:* como memória **virtual**, paginação com LRU (sem fragmentação externa). Deixe claro que é virtual.

Alternativa batch (se preferir): batch, multiprogramado, contíguo, hierárquica; FCFS/SJF não preemptivo (maximiza o throughput); deadlock por pré-alocação declarada no JCL (aceitável em batch; reconheça que reduz o throughput); starvation: FCFS não tem, SJF exige aging; memória real: **partições** (fixas com fila por partição, ou variáveis com primeiro ajuste).

Armadilhas que custam ponto:

- "Liberar recursos quando não precisar mais" **não** evita deadlock (foi o seu erro).
- "Escalonar por prioridade" **causa** starvation; só evita com aging (foi o seu erro).
- Round-robin evita starvation, **não** deadlock.
- Paginação/segmentação **não** são gerência de memória **real** nos slides.
- Não esquecer a **estrutura** (você esqueceu).

Nota sobre os guias individuais: o modelo A do guia 01 combina carregamento **não contíguo** com swapping (que na tabela de 8-2 s15 é contíguo). É defensável, mas menos coerente que o modelo acima. O guia 03 indicava paginação como "memória real", o que foi corrigido (ver seção 8).

4.5 Perguntas dos slides que podem virar discursiva (não caíram; respostas curtas)

Respostas montadas a partir dos slides, sem confirmação em prova; confiança média.

- **Problema do carregador de relocação (7-2 s49):** exige bloco **contíguo** do tamanho do programa → fragmentação; programa maior que o maior bloco livre não carrega; depois de carregado não pode ser movido (a relocação é estática). Isso motiva o registrador de base e a paginação.
- **Operador para p e d (8-2 s21):** concatenação (página potência de 2, frames alinhados).
- **Por que a paginação facilita a proteção (8-2 s20):** cada processo só alcança os frames da sua tabela de páginas, e cada entrada pode ter bits de proteção.
- **Quantos buffers (7-1 s8, s12):** dois ou mais (double buffering): enquanto um é processado, o outro é preenchido pela E/S.
- **Efeito de um desvio incondicional (7-1 s7):** o PC recebe o endereço de destino em vez de ser incrementado; quebra a sequência (e a localidade espacial) e esvazia o pipeline.
- **Reconhecimento da fonte de interrupção (7-1 s9; 8-1 s8):** a CPU propaga o ACK pelas fontes em ordem de prioridade; a fonte que interrompeu coloca sua identificação no barramento de dados, e a CPU usa essa identificação como índice no **vetor de interrupções**.
- **Referência futura e número de passos (7-1 s18):** símbolo usado antes de ser definido. O montador de passo único precisa de uma lista de pendências para corrigir depois; o de dois passos monta a tabela de símbolos no passo 1 e gera o código no passo 2 (o típico hoje: é mais simples e a memória é barata).
- **Requisito de memória do overlay (8-2 s16):** fase raiz + o caminho de fases mais "caro" da árvore (a maior soma raiz → folha).

5. Seus pontos fracos (a partir da sua Prova-T 21/2)

Estimativa da sua Parte 1: cerca de 5,8 a 6,2 de 7,0 (20 acertos, 1 erro, 1 NS se a Q6 valer; 19 acertos e 2 erros se não valer). As discursivas foram onde você mais perdeu.

#	O que aconteceu	Conceito correto	Revisar
Q14	Marcou **F** ("a cada busca, PC é incrementado")	Omitir um detalhe não torna o item falso; o item não afirma nada errado → **V** provável	Critério 3.0
Q6	Marcou "**F/NS**" com justificativa "a otimização algorítmica pode ser feita pelo montador"	Marcação dupla é inválida; a justificativa contradiz 7-1 s31: a síntese otimiza o **código intermediário**, **não** o algoritmo → F	3.2(c)

#	O que aconteceu	Conceito correto	Revisar
Q15	NS	Bytecode Java não é código objeto relocável; a portabilidade é da JVM → F	3.4(b)
Q8, Q11	Justificativas certas, mas genéricas/curtas ("Sistema operacional do tipo batch")	Cite o fato do slide: "ainda usado no boot (ROM BIOS)"; "time sharing é interativo/E/S; a gerência é o swapping"	3.4(a), 3.5(b)
D1	Repetiu os tipos (temporal/espacial) como "causa"	Causas = armazenamento adjacente + execução sequencial + laços + vetores + pilha	4.1
D2	Faltou o **instante** da relocação	Absoluto: nenhum; relocação: na carga (antes da execução); DLL: na execução	4.2
D3	"Endereço real = lógico + RegBase", sem exemplo	Isso é o registrador de base. Paginação: (p, d) → RegBase + p → frame → **r ‖ d** + exemplo numérico	4.3 (treine o exemplo à mão)
D4	Sem estrutura; "liberar recursos" para deadlock; "prioridade" para starvation	Estrutura hierárquica; ordenação de recursos (espera circular); round-robin + aging	4.4

Hábitos a corrigir:

1. Transforme cada discursiva num checklist dos pedaços pontuados e responda cada um com subtítulo.
2. Na D3, um exemplo numérico vale metade da questão: treine-o até sair de cabeça.
3. Para marcar F, aponte uma afirmação **errada**, não algo que faltou (exceto lista definidora incompleta).
4. Uma marcação por item.

6. Padrões de pegadinha do professor

1. **Cópia do slide com uma palavra trocada** (o mais comum):

Palavra certa	Troca usada
não volátil	volátil
relocável	absoluto
executável	não executável
CPU × memória	CPU × E/S
segmentos (fragmentação externa)	páginas
starvation	deadlock
batch (CPU intensiva)	time sharing
memória virtual	memória real
otimização de código	otimização algorítmica

2. **Primeira parte verdadeira + conclusão ou exemplo falso:** "por isso deve ser não preemptivo"; "como é o caso do bytecode Java"; "para maximizar o throughput **e** evitar deadlock".
3. **Lista definidora com um item a menos:** von Neumann sem barramentos. Decore as listas completas: 4 componentes; 4 partes da CPU; 3 barramentos; 4 etapas do ligador; 3 tipos de ligador; 3 técnicas de E/S no gargalo.
4. **Pares de itens com associações trocadas:** batch ↔ time sharing com partições ↔ paginação-segmentação (P1-Q19/20, PT-Q10/11).
5. **Palavras absolutas e avaliativas:** "apenas", "sempre", "deve ser", "melhor", "não é mais utilizado". Elas pedem verificação, mas **não** tornam o item falso automaticamente: o "apenas" da P1-Q13 é V, porque o slide lista exatamente aqueles tipos. Já "deve ser não preemptivo" e "não é mais utilizado" são F.
6. **Atribuir a função ao programa errado:** montador × ligador × carregador × S.O. (quem resolve referências externas: o ligador; quem faz PC ← entry point: o carregador; quem move de bloqueado para pronto: a interrupção; quem escolhe quem executa: o escalonador).
7. **Causalidade levemente invertida mas copiada do slide** ("devido ao fenômeno de localidade"): fica V.
8. **Mesmo item em outra ordem, entre provas:** reconheça pelo conteúdo, não pelo número.

7. Folha de revisão rápida (cola mental)

von Neumann

- 4 componentes: **CPU (ULA, UC, registradores, PC), memória (dados e programas), E/S (pode ser mapeada na memória), barramentos (dados, endereço, controle)**.
- Ciclo: busca → decodificação → busca de operandos → execução. **PC ← entry point** (pelo carregador); PC incrementado a cada busca.
- Execução rápida: dados e instruções em blocos adjacentes → localidade de referência.
- Gargalo CPU×memória: **cache, pipelining**. Gargalo memória×E/S: **blocagem/bufferização, DMA, canais**.
- E/S por interrupção: bloqueia → supervisor cria **programa de canal** → canal executa → **interrupção** → controlador desbloqueia → fila de prontos.

Código e carregadores

- Código absoluto: endereços fixos e reais; **ainda usado no boot** (ROM BIOS, FFFF:0000 tem um jmp).
- Relocação estática: endereço = offset + α (constante k); **viabiliza bibliotecas de código objeto**; feita antes da execução (o carregador soma α na carga).
- Carregador absoluto: inicia a carga do S.O.; código absoluto; **memória não volátil** (ROM/flash), posição fixa; botão; **IPL** no mainframe; registro = endereço de carga, número de palavras, dados; n = 0 → transferto.
- Carregador de relocação: tamanho + entrypoint → pede **bloco contíguo** ao S.O. → carrega relocando → PC ← entrypoint. Problema: contiguidade/fragmentação.
- Boot (slide 9): Power Good → ROM BIOS (FFFF:0000) → POST (cold start completo; warm start se 0000:0472 = 1234h, sem teste de memória) → CMOS dá a ordem dos drives → **MBR** (cilindro 0, cabeça 0, setor 1; 512 bytes) carregado em **0000:7C00**, assinatura **55AAh** → boot loader → registro de boot da partição ativa → **NTLDR** (modo real → protegido) → BOOT.INI → NTDETECT.COM → ntoskrnl.exe + hal.dll.

Montador

- Assembly → **objeto relocável** + tabela de relocáveis + lista de referências externas. Com erro de sintaxe, não gera código.
- Lógica: pré-passo (macros, montagem condicional, inclusões, tira comentários) → léxica → sintática → semântica (código intermediário) → **síntese** (otimização **do código**, opcional, + geração de **relocável**).

- Física: passo único (problema da referência futura) × múltiplos passos (pré-passo, passo 1 com a tabela de símbolos, passo n, passo final).

Ligador

- Resolve referências externas; entrada: objetos relocáveis; saída: **executável relocável** + endereços relocáveis.
- Etapas: **ler/relocar P → tabela global com os símbolos internos (públicos) → concatenar → resolver as externas**. k de P2 = size(P1).
- Dois passos: 1 = relocação + tabela global (+ checa o entry point); 2 = resolve as referências externas.
- Tipos: **Linkage Editor** (gera módulo de carga reutilizável) · **Linking Loader/Binder** (carrega direto, sem arquivo) · **Dynamic linking/DLL** (em execução, compartilhada, *fixup* atualizado pelo carregador; .dll no Windows, lib*.so no Linux).

S.O. e memória

- Tipos: single-job (CP/M, DOS), multiprogramado (IBM VM), multiprocessado (Solaris, z/OS), rede (NetWare), distribuído (Amoeba), **batch** (lote, MVS), **time sharing** (time slice, TSO), tempo real (QNX, FreeRTOS).
- Estruturas: hierárquica (kernel no nível 0), máquinas virtuais (IBM VM), multiprocessados (fortemente ou fracamente acoplados), distribuídos, orientados a objetos.
- Memória **real**: overlay, partições, swapping, registrador de base (ER = EL + RegBase via DAT). **Virtual**: paginação, segmentação, paginação-segmentação.
- Blocos fixos → sem fragmentação (externa); variáveis → combater com pior, melhor ou primeiro ajuste (o primeiro permite o colapso de blocos); combater a fragmentação exige realocação.
- Paginação: fixa; só a página do entrypoint é carregada no início; DAT + tabela; **LRU** (bit de referência zerado periodicamente); **concatenação**; interna sim, externa não.
- Segmentação: variável; **soma**; externa sim; "facilita em muito a proteção"; custo maior.
- Batch ↔ CPU intensivo ↔ partições. Time sharing ↔ interativo ↔ swapping (time slice, escalonamento por prioridade, troca leva milissegundos).

Processos

- Estados: pronto, executando, bloqueado, finalizado; contexto salvo no **PCB**.
- Objetivos: **maximizar throughput; minimizar turnaround e response time**; pode ser preemptivo ou não.
- **Round-robin evita starvation, não deadlock.** Prioridade sem aging causa starvation.
- Deadlock: 4 condições (exclusão mútua, posse e espera, não preempção, espera circular). Pré-alocação quebra a posse e espera, mas reduz o throughput; ordenação de recursos quebra a

espera circular.
- Região crítica: trecho/mecanismo de exclusão mútua; sem ela → **condição de corrida** (não deadlock).
- Interrupção: comuta usuário → supervisor; fontes: software (SVC/INT), condição de programa (divisão por zero), hardware; vetor de interrupções; PSW guarda o contexto.

8. Divergências e incertezas

8.1 Itens com resposta não segura

Item (P1 / PT)	Resposta recomendada	Confiança	Por quê
Ciclo + entry point (Q2 / Q14)	**V**	média (~65%)	Nada no item é falso; a omissão do incremento do PC não é de lista definidora (critério 3.0). Dois alunos marcaram F; os três guias, V.
von Neumann (Q1 / Q1)	**F**	média-alta (~80%)	Lista "caracterizada por" sem os barramentos + "RAM".
Bytecode Java (Q6 / Q15)	**F**	média (~70%)	Exemplo errado: a portabilidade vem da JVM. Os três guias concordam.
Otimizações "algorítmicas" (Q10 / Q6)	**F**	média (~70%)	7-1 s31: "a otimização é do código e não das funções do programador". Os três guias concordam.
Região crítica (Q18 / Q17)	**V**	média (~60%)	Terminologia do professor (8-1 s10) e consenso dos três alunos; a definição dos livros sugeriria F.
Tipos de ligador (Q13 / Q13)	**V**	média-alta (~75%)	Os slides listam exatamente os 3 tipos; risco no "apenas" e em "carregador de função".
Batch + paginação-segmentação (Q19 / Q10)	**F**	média-alta (~75%)	Paginação-segmentação é memória **virtual** nos slides; batch casa com partições.

Item (P1 / PT)	Resposta recomendada	Confiança	Por quê
Uma palavra: bloqueado → pronto (extra / –)	**Interrupção**	média-alta (~80%)	7-1 s9–s10. "Escalonador" atua de pronto → executando.
Localidade "devido ao fenômeno" (Q3 / Q2)	**V**	alta (~85%)	Cópia do slide, apesar da causalidade invertida.
Segmentos + fragmentação (Q21 / –)	**V**	alta (~85%)	8-2 s22 + 8-1 s13. O aluno de P1 errou (F).

8.2 Divergências entre os guias e como foram resolvidas

Ponto	Guia(s)	Divergência	Resolução
Critério de omissão (Q1 × Q14)	01, 02, 03	Pareciam critérios contraditórios	Critério único (3.0): enumeração definidora incompleta → F; descrição correta sem um detalhe → V. Confirmado pelo item do carregador de relocação (incompleto e V para todos).
D4: gerência de memória **real**	03 × 01/02	O 03 indicava **paginação** como memória real (e swapping só como alternativa)	**Erro claro**, contraria 8-2 s20–s23 e a própria justificativa da Q10 do 03. **Corrigido no arquivo 03**: swapping como resposta, paginação como complemento de memória virtual.

Ponto	Guia(s)	Divergência	Resolução
D4: carregamento	01 (não contíguo) × 02 (contíguo)	Com swapping como memória real, "contíguo" é o coerente com 8-2 s15	Recomendado **contíguo + swapping + registrador de base**. O 01 não foi alterado: é defensável, só menos coerente.
Pergunta de uma palavra	01 ("Interrupção") × 02 ("controlador de interrupções")	Mesmo mecanismo	Escrever **Interrupção** (uma palavra).
Estrutura do montador (Q9)	03 aponta um erro secundário (lógico × físico); 01 diz que a estrutura está certa	Não muda a resposta	**F** pelo "absoluto"; não use o argumento lógico × físico como justificativa principal.
Exemplo da D3	01 (frame 5 → 0x5ABC) × 03 (frame 7 → 0x7ABC)	Tabelas diferentes, os dois corretos	Use um só (seção 4.3).
Itens correspondentes P1 × PT	todos	Respostas diferentes em ligador (executável), round-robin (starvation/deadlock) e fragmentação (segmentos/páginas)	Todas as diferenças vêm de **enunciado diferente**, e não de erro de agente. Nos demais pares (bytecode, otimização, batch/time sharing, região crítica, pré-alocação etc.) os três guias dão a **mesma** resposta.

8.3 Erros dos alunos (não dos guias) que vale lembrar

- Aluno de P1: Q21 (segmentos) marcou F → é **V**; Q6 (bytecode) e Q10 (otimização) marcou V → provável **F**; extra "Escalonador" → **Interrupção**; justificativa da Q4 conceitualmente errada.

- Outro aluno da PT: Q6 e Q12 marcou V → **F**; justificativa da Q17 ("deadlock") → é condição de corrida; justificativa da Q20 ("RR não é justo") → errada.
- Você: ver seção 5.